BabyVision Experiments: RL Fine-Tuning of Gemma-4 for Visual Reasoning

Yoav Sinai

Contents

1 Abstract **1**

2 Introduction **2**
2.1 Project Structure 2
2.2 Codebase Provenance 3

3 Method **4**

4 Identification and Resolution of an Evaluation Defect **4**

5 Results **5**
5.1 Final Results (1024-token budget) 5
5.2 Category-Level Results 6
5.3 Subtype-Level Results (Baseline vs. Best RL Configuration) 6

6 Discussion **7**
6.1 Mechanism Underlying the Configuration C Improvement 7
6.2 The Generation Token Budget as a Confounding Variable 8
6.3 Implications for BAGEL Evaluation 9

7 Conclusion **9**

1 Abstract

This report documents a seminar project built on top of the BabyVision benchmark, a visual reasoning benchmark of 388 puzzle-style tasks spanning fine-grained discrimination, spatial perception, visual tracking, and pattern recognition. The original paper demonstrates that reinforcement learning (RL) fine-tuning can improve a model's visual reasoning performance on this benchmark. This project applies that same RL recipe to `google/gemma-4-E4B-it`, a model not covered in the original paper, and evaluates whether the approach transfers.

The best configuration (multimodal LoRA targets with an increased correctness reward weight) improved accuracy from a 12.97% baseline to 13.75%. During the course of the project, an evaluation-script defect was identified and corrected: Gemma's generation was being capped at a substantially smaller token budget than other models under evaluation, causing a large share of its answers to be truncated before completion. Correcting this budget, first to 512 and then to 1024 tokens, revealed that an earlier finding in the project

("Gemma clearly outperforms Qwen") was itself almost entirely an artifact of this truncation bug rather than a genuine difference in model capability. The token generation budget proved to be the single most influential variable in this project, with a larger effect on measured accuracy than any of the RL training design choices evaluated.

2 Introduction

The objective of this project was to evaluate whether RL fine-tuning, as proposed in the BabyVision paper, generalizes to a model not evaluated in the original work. `google/gemma-4-E4B-it` was selected as the target model. `Qwen/Qwen2.5-7B-Instruct` was used throughout as an automated judge, comparing each model's extracted answer against the ground truth. A stock, untrained `Qwen/Qwen2.5-VL-7B-Instruct` was evaluated in parallel as a baseline comparison point.

All results reported below are based on a small-sample evaluation protocol (388 tasks, 3 passes per configuration) appropriate for a seminar-scale investigation. The results should be interpreted as directional evidence of what helps, not as a statistically rigorous benchmark claim.

2.1 Project Structure

This section is a navigation aid to the repository, oriented toward finding this report's supporting evidence rather than describing every file. Paths are relative to the repository root.

```
BabyVision/
  results/
    final_report.pdf                          <- this report
    accuracy_comparison.png                   <- chart used in Results

  babyvision_eval/                            <- MLLM (text-answering) track
    evaluate_model.py, compute_score.py,      <- original fork (API-based only)
    utils.py
    evaluate_local_hf.py                      <- local-GPU evaluation (Gemma + Qwen),
                                                 added this project
    evaluate_system_prompt.py                 <- two-stage system-prompt evaluation
    rl_train_gemma4.py                        <- RL training, Configuration B
    rl_train_gemma4_correctness_weight2.py    <- RL training, Configuration C (best)
    rl_train_system_prompt.py                 <- RL training, Configuration D
    runs/                                     <- evaluation outputs, one subfolder per
                                                 configuration x token budget, e.g.
                                                 runs/rl_correctness_weight2_maxtok1024/
                                              (model_results_run_*.json, score_summary.txt)

  babyvision_gen_eval/                        <- generation (image-editing) track
    scripts/
      inference_bagel.py                      <- BAGEL native image-editing inference
      evaluate_local_hf.py                    <- local-GPU judge for this track
   generated/bagel_find_different/round1/     <- BAGEL outputs, Find-the-different test
    results/
     bagel_find_different/round1/eval.jsonl   <- judged per-task results for that test
      summary.txt, summary.json               <- full 280-task run, all models
```

```
    summary_find_different.txt/.json      <- dedicated summary for the 16-task test

  sbatch_scripts/
    training/, evaluation/, other/          <- one SLURM script per job referenced
                                               in Appendix A

  logs/
    training/, evaluation/, setup/, archive/ <- SLURM stdout/err, named by job ID
                                                (see Appendix A for the mapping)

  data/
    babyvision_data/meta_data.jsonl         <- the 388-task MLLM benchmark
    babyvision_gen_data/                    <- the generation-track benchmark
```

Every job ID referenced in this report (Appendix A) can be traced to its exact stdout/err pair under `logs/`, and every accuracy number in Section 5 traces back to the raw per-task JSON under the corresponding `babyvision_eval/runs/*/` directory.

2.2 Codebase Provenance

The forked repository (`babyvision_eval/`) originally provided only an API-based evaluation script (`evaluate_model.py`), a scoring utility (`compute_score.py`), and shared helper functions (`utils.py`). No local-GPU evaluation path and no RL training code existed in the original fork. The following components were implemented for this project (paths relative to `babyvision_eval/`, except the final two rows, which are under `babyvision_gen_eval/scripts/`):

Script	Purpose
`evaluate_local_hf.py`	Local-GPU evaluation for Gemma and Qwen (the upstream script supports only API-based inference). This is where the token-budget defect described below originated.
`evaluate_system_prompt.py`	Two-stage evaluation for the system-prompt RL variant: one model call generates a system prompt, and a second, frozen call answers the question using that prompt.
`rl_train_gemma4.py`	GRPO training with correctness_weight=1.0 and language-model-only LoRA targets (Configuration B).
`rl_train_gemma4_correctness_weight2.py`	GRPO training with correctness_weight=2.0 and multimodal LoRA targets (Configuration C, the best-performing setup).
`rl_train_system_prompt.py`	GRPO training for the system-prompt-generating policy (Configuration D).

Script	Purpose
`inference_bagel.py`	Runs ByteDance's BAGEL-7B-MoT in its native image-editing mode on the generation track. An earlier iteration of this project evaluated BAGEL through the text-answering understanding track instead, which was determined to be an inappropriate benchmark for this model; see the BAGEL discussion below and the deprecated understanding-track notes for details.
`evaluate_local_hf.py` (generation track)	Local-GPU judge for the generation track (the upstream `evaluate.py` supports only API-based judging). This script is model-agnostic by design and required no modification to judge BAGEL's output.

Two components developed during the project did not contribute to the final results and are archived rather than presented as part of the active pipeline: `babyvision_eval/_deprecated/test_peft.py` (a diagnostic script verifying which model layers a given LoRA target-module pattern matches, never invoked by any training or evaluation run) and `babyvision_gen_eval/_deprecated/` (an earlier generation-track attempt using `timbrooks/instruct-pix2pix`, superseded by the BAGEL-based approach and not referenced in any reported result).

3 Method

Four training/evaluation configurations were evaluated in sequence, in addition to an untrained baseline:

- **Configuration A (Baseline).** Stock Gemma, no training.
- **Configuration B (RL, correctness_weight=1.0).** GRPO training with LoRA adapters applied only to language-model layers.
- **Configuration C (RL, correctness_weight=2.0).** GRPO training with two modifications relative to Configuration B: LoRA adapters were extended to the multimodal connector and projection layers, and the correctness component of the reward was doubled, reducing the relative incentive to satisfy the output-format reward without producing a correct answer.
- **Configuration D (System-prompt RL).** Rather than training the model to answer directly, this configuration trains the model to generate a system prompt for itself, on the hypothesis that a self-generated instruction could improve downstream answer quality.

4 Identification and Resolution of an Evaluation Defect

During initial analysis, the seminar instructor observed that the RL results appeared anomalous. Investigation of the evaluation pipeline identified a defect in `evaluate_local_hf.py`: Gemma's generation was capped at 256 tokens, while every other model under evaluation received a 512-token budget. This asymmetry caused a substantial fraction of Gemma's answers to be truncated before the model could produce a final answer within the required `\boxed{...}` format. The defect was corrected by applying a uniform 512-token budget across all models, and every configuration was re-evaluated. All configurations showed improved accuracy following the correction (see Results below).

Further analysis established that 512 tokens remained insufficient. The rate of non-extractable ("no-answer") responses was measured across all configurations at the 512-token budget: Gemma variants exhibited a 16.7-21.1% no-answer rate (Configuration C excepted, which stayed below 1%), while the untrained Qwen baseline exhibited an 81.4% no-answer rate. This indicated that the previously reported finding of Gemma outperforming Qwen was itself substantially confounded by the same truncation effect, to a more severe degree. The token budget was subsequently increased to 1024 tokens for all configurations.

At 1024 tokens, Qwen's no-answer rate dropped to 1.9%, and its accuracy increased from 2.84% to 13.32%, a result approximately equivalent to the best-performing Gemma RL configuration. The apparent performance gap between Gemma and Qwen identified earlier in the project is attributable to an inconsistent token budget rather than a genuine difference in underlying model capability.

5 Results

5.1 Final Results (1024-token budget)

Configuration	Accuracy	No-answer rate
A: Baseline Gemma	12.97% ± 0.99%	6.6% ± 0.2%
B: RL (correctness_weight=1, language-only LoRA)	11.34% ± 0.56%	6.7% ± 0.6%
C: RL (correctness_weight=2, multimodal LoRA)	**13.75% ± 0.49%**	0.3% ± 0.2%
D: System-prompt RL	13.66% ± 0.56%	8.3% ± 1.5%
Qwen baseline (untrained)	13.32% ± 0.44%	1.9% ± 0.3%

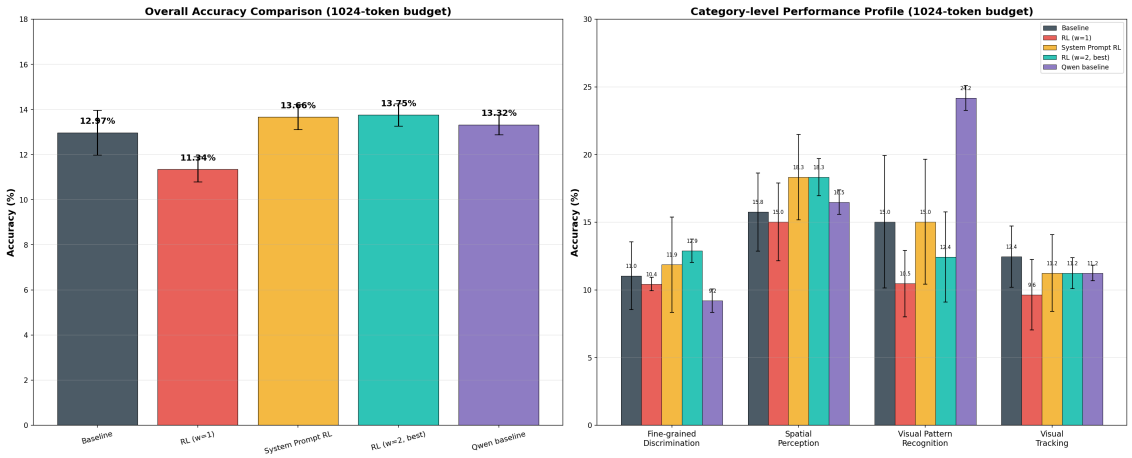
Several observations follow from these results:

- Configuration C and the Qwen baseline are approximately tied for the best result, with Configuration D close behind. The range between the best and worst configuration (11.34% to 13.75%) is considerably narrower than it appeared at earlier, truncation-affected token budgets, indicating that a portion of the apparent differences between configurations reflected differential sensitivity to truncation rather than differences in visual reasoning capability.
- Configuration B performs marginally worse at 1024 tokens than it did at 512 tokens (12.20% versus 11.34%). No definitive explanation for this regression was established; it may reflect sampling variance given the limited evaluation set (388 tasks, 3 passes), or a behavioral characteristic of this specific checkpoint under a larger generation budget. This result is reported without further interpretation, given the available evidence.
- Configuration C maintains the lowest no-answer rate of any configuration at every token budget tested, a consistent effect attributable to the increased correctness-reward weighting, which appears to train the model toward producing complete, concise answers.
- With a uniform 1024-token budget and no training applied to either model, Gemma (12.97%) and Qwen (13.32%) achieve comparable accuracy. Neither model demonstrates a clear advantage on this benchmark under equivalent evaluation conditions.
- Configuration D's result (13.66%) is close to Configuration C's despite a different architectural rationale (a self-generated system prompt rather than a directly reward-shaped answer policy). One spec-

ulative explanation, offered here only as a hypothesis and not as a conclusion this project supports, is that Configuration D's improvement over baseline reflects the additional inference-time compute from its two model calls per task rather than a benefit specific to the system-prompt-generation architecture itself. This project does not include a compute-matched control that would isolate the two explanations, so no claim is made either way.

5.2 Category-Level Results

Category	Baseline	RL (B)	RL (C)	System-prompt RL (D)	Qwen
Fine-grained Discrimination	11.04%	10.43%	12.88%	11.86%	9.20%
Spatial Perception	15.75%	15.02%	18.32%	18.32%	16.48%
Visual Pattern Recognition	15.03%	10.46%	12.42%	15.03%	**24.18%**
Visual Tracking	12.45%	9.64%	11.24%	11.24%	11.24%



Qwen's Visual Pattern Recognition result (24.18%) is the strongest single-category result across all configurations evaluated and warrants further investigation beyond the scope of this project.

5.3 Subtype-Level Results (Baseline vs. Best RL Configuration)

Category	Subtype	Baseline	RL (C)
Fine-grained Discrimination	2D Pattern Completion	33.33% ± 6.24%	**45.00% ± 4.08%**
Fine-grained Discrimination	Count Clusters	11.11% ± 4.54%	**12.96% ± 2.62%**
Fine-grained Discrimination	Find the Shadow	5.80% ± 4.10%	**7.25% ± 2.05%**

Category	Subtype	Baseline	RL (C)
Fine-grained Discrimination	Pattern and Color Completion	30.00% ± 8.16%	**33.33% ± 2.36%**
Fine-grained Discrimination	Find the same	1.96% ± 2.77%	1.96% ± 2.77% (tie)
Visual Pattern Recognition	Rotation Patterns	23.33% ± 12.47%	**26.67% ± 20.55%**
Visual Pattern Recognition	Logic Patterns	**4.76% ± 6.73%**	2.38% ± 3.37%
Visual Pattern Recognition	Overlay Patterns	**21.57% ± 7.34%**	15.69% ± 2.77%
Spatial Perception	3D Pattern Completion	33.33% ± 9.07%	**46.30% ± 6.93%**
Spatial Perception	Paper Folding	13.89% ± 10.39%	**16.67% ± 11.79%**
Visual Tracking	Connect the lines	7.02% ± 2.48%	**15.79% ± 0.00%**
Visual Tracking	Maze	**20.00% ± 4.08%**	18.33% ± 2.36%
Visual Tracking	Recognize numbers/letters	**18.84% ± 4.10%**	11.59% ± 2.05%

The largest gains for Configuration C over baseline occurred in 3D Pattern Completion (33.33% → 46.30%) and Connect the Lines (7.02% → 15.79%). The most notable regressions occurred in Recognize numbers/letters (18.84% → 11.59%) and Overlay Patterns (21.57% → 15.69%). The single largest subtype-level regression for Configuration C overall was 3D Cube Unfold, which declined from 5.56% at baseline to 0.00%.

Raw per-task outputs for all configurations, all passes, are available in `model_results_run_*.json` under each configuration's output directory, for example `babyvision_eval/runs/rl_correctness_weight2_maxtok1024/`.

6 Discussion

6.1 Mechanism Underlying the Configuration C Improvement

The improvement from Configuration B to Configuration C is attributable to two concurrent changes, both of which appear necessary:

- **Multimodal LoRA target modules.** Configuration B applied LoRA adapters only to language-model layers. The available evidence is consistent with this producing a representational drift between the model's text output and the frozen vision encoder's representations, since only the language-side parameters were updated. Extending LoRA to the multimodal connector and projection layers in Configuration C appears to preserve this alignment.
- **Correctness reward weighting.** Under correctness_weight=1.0, the formatting component of the reward (correct use of `\boxed{...}`) was inexpensive to satisfy relative to producing a correct answer, permitting the policy to accumulate reward through format compliance with limited regard for correctness. Doubling the correctness weight in Configuration C shifted a larger proportion of the gradient signal toward answer correctness.

These findings suggest that GRPO training improves visual reasoning performance on this benchmark under two conditions: LoRA adaptation must extend to the multimodal connector layers, and the correctness reward must be weighted sufficiently to prevent the policy from optimizing primarily for output format.

6.2 The Generation Token Budget as a Confounding Variable

Table 1 below summarizes accuracy and no-answer rate across all three token budgets evaluated during this project.

Table 1: Accuracy and no-answer rate by token budget.

Configuration	Acc. (256 tok)	Acc. (512 tok)	Acc. (1024 tok)	No-answer (256)	No-answer (512)	No-answer (1024)
A: Baseline Gemma	11.08%	12.03%	12.97%	24.4% ± 0.5%	16.8% ± 2.1%	6.6% ± 0.2%
B: RL (weight=1)	9.54%	12.20%	11.34%	27.2% ± 1.5%	16.7% ± 1.4%	6.7% ± 0.6%
C: RL (weight=2)	13.14%	13.66%	13.75%	2.1% ± 0.4%	0.7% ± 0.3%	0.3% ± 0.2%
D: System-prompt RL	9.71%	10.48%	13.66%	34.5% ± 1.8%	21.1% ± 2.3%	8.3% ± 1.5%
Qwen baseline	-	2.84%	**13.32%**	-	81.4% ± 0.4%	1.9% ± 0.3%

The pattern is consistent across all configurations: an increase in the token budget is associated with a decrease in the no-answer rate and, correspondingly, an increase in measured accuracy. The Qwen baseline represents the clearest evidence that this relationship is causal rather than incidental: its no-answer rate decreased from 81.4% to 1.9% between the 512- and 1024-token conditions, and its measured accuracy correspondingly moved from substantially below every Gemma configuration to approximately matching the best-performing configuration. This result is not consistent with an improvement in the underlying model's reasoning capability between conditions; the model was unchanged. It is consistent with the model being given sufficient generation budget to complete an answer it was already capable of producing.

This finding directly accounts for the observation that initiated this investigation. At a 256-token budget, Configuration B appeared to underperform the baseline by a substantial margin (9.54% versus 11.08%). At a 1024-token budget, with truncation minimized for both configurations, this gap narrows to 11.34% versus 12.97%, a smaller but still present difference. The principal methodological conclusion of this project is that comparison between models or training configurations on a free-form generation task is valid only when the token budget is held constant across all conditions and is sufficiently generous that truncation does not materially affect any condition under comparison. In this project, the token budget setting had a larger effect on measured outcomes than any of the RL training design choices evaluated.

This defect was confined to evaluation and did not affect training. The RL training scripts (`rl_train_gemma4.py`, `rl_train_gemma4_correctness_weight2.py`) generated rollouts with `max_completion_length=256`, and `rl_train_system_prompt.py` used `max_completion_length=128`, uniformly across all training runs regardless of configuration. This was a deliberate training-efficiency budget, not an instance of the same asymmetry bug, since every training configuration used the same cap. Whether training rollouts were themselves frequently truncated at this budget, and whether that would have affected the quality of the learned policy, was not investigated and is noted here as an open question for follow-up work.

6.3 Implications for BAGEL Evaluation

ByteDance's BAGEL-7B-MoT was evaluated on the generation track because the original BabyVision paper cites it by name as an example of unified understanding-and-generation models that "offer a new path forward" for solving tasks through image-space reasoning rather than text. A full 280-task, multi-round evaluation of BAGEL was judged impractical given the compute cost of running its generation pipeline at scale, so the evaluation was instead narrowed to the single hardest available case: the subtype "Find the different," the only subtype on which every other evaluated model in this project (Configurations A through D and the Qwen baseline) scored exactly 0%. BAGEL was evaluated on the 16 available tasks of this subtype using its native image-editing capability (marking the target region directly on the input image, rather than producing a text answer), obtaining 1 correct response of 16 (6.25%). While this sample is too small to support a general conclusion, it is a nonzero result on a subtype where every text-based configuration evaluated in this project scored zero, and is consistent with the hypothesis that image-space reasoning may be more appropriate for certain BabyVision task types than text-based reasoning. Given the small sample size, this result should be treated as a preliminary observation rather than a validated finding.

This generation-track evaluation followed an earlier, incorrect attempt: BAGEL was initially evaluated through the text-answering understanding track, producing an overall score of 4.12% with a 78% rate of responses truncated before completion of the model's internal reasoning process. That evaluation is considered to have targeted an inappropriate benchmark for a model of this architecture, since BAGEL's relevant capability is native image editing rather than text-based answering, and is retained for reference in `babyvision_eval/_deprecated/bagel_understanding_track/` but is not treated as part of the primary results of this project.

7 Conclusion

RL fine-tuning of `google/gemma-4-E4B-it` using the BabyVision paper's recipe produced a modest but consistent improvement over baseline (13.75% versus 12.97%), attributable to the combination of multimodal LoRA targeting and increased correctness reward weighting. A secondary but methodologically significant finding of this project is that the generation token budget used during evaluation was a substantial confounding factor throughout, sufficient to account for the majority of an earlier, incorrect conclusion that Gemma clearly outperformed Qwen on this benchmark. Under a corrected, uniform evaluation protocol, Gemma and Qwen perform comparably as untrained baselines, and the RL-tuned Gemma configuration and the untrained Qwen baseline are approximately tied for the best overall result obtained in this project.

Appendix A: Job IDs and Logs (for reproducibility)

Run	Job ID	Logs
Baseline eval (256 tok, original)	`16875224`	Out / Err
RL training, w=1	`16875257`	Out / Err
RL eval, w=1 (256 tok, original)	`16876673`	Out / Err
RL training, w=2	`16877068` (resumed `16877168`)	Out / Err
RL eval, w=2 (256 tok, original)	`16877072`	Out / Err
System-prompt RL training	`16877070`	Out / Err

Run	Job ID	Logs
System-prompt RL eval (256 tok, original)	`16877197`	Out / Err
Baseline eval, 512 tokens	`23604080`	Out / Err
RL eval, w=1, 512 tokens	`23657919`	Out / Err
RL eval, w=2, 512 tokens	`23657920`	Out / Err
System-prompt RL eval, 512 tokens	`23657921`	Out / Err
Qwen baseline eval, 512 tokens	*(archived, no job ID retained)*	-
Baseline eval, 1024 tokens	`23671051`	Out / Err
RL eval, w=1, 1024 tokens	`23671052`	Out / Err
RL eval, w=2, 1024 tokens	`23671053`	Out / Err
System-prompt RL eval, 1024 tokens	`23671054`	Out / Err
Qwen baseline eval, 1024 tokens	`23671055`	Out / Err
BAGEL “Find the different” eval, gen track	`23820802`	Out / Err